

Universita degli Studi di Messina
Department of Mathematical, Computer, Physical and Earth Sciences

MetaChat

Extended Triple Diffie-Hellman & Double Ratchet in Go

University Project Report

Creators	Mehdi Talebikhatir (ID: 558948) Benyamin Baharizadeh (ID: 560587)
Repository	https://github.com/mehditalebi01/metachat
Main purpose	Educational end-to-end encrypted messaging in Go
Date	April 27, 2026

Abstract / Executive Summary

MetaChat is a small secure messaging project written in Go. It shows how two users can exchange an encrypted message through a server that does not know the plaintext. The project uses X25519 Diffie-Hellman, HKDF-SHA256, a Double Ratchet-inspired key chain, AES-256-GCM, and Redis.

The implementation is educational. It studies the ideas behind Extended Triple Diffie-Hellman and the Double Ratchet, but it is not a complete production Signal Protocol implementation. In the current code, Benny publishes one X25519 public key, Matteo creates a fresh ephemeral key, both sides derive the same message key, and the server stores only public keys, nonces, and ciphertext.

Introduction

Secure messaging is important because a normal server-based chat gives too much power to the server. End-to-end encryption avoids this by keeping the message content encrypted until it reaches the receiver device.

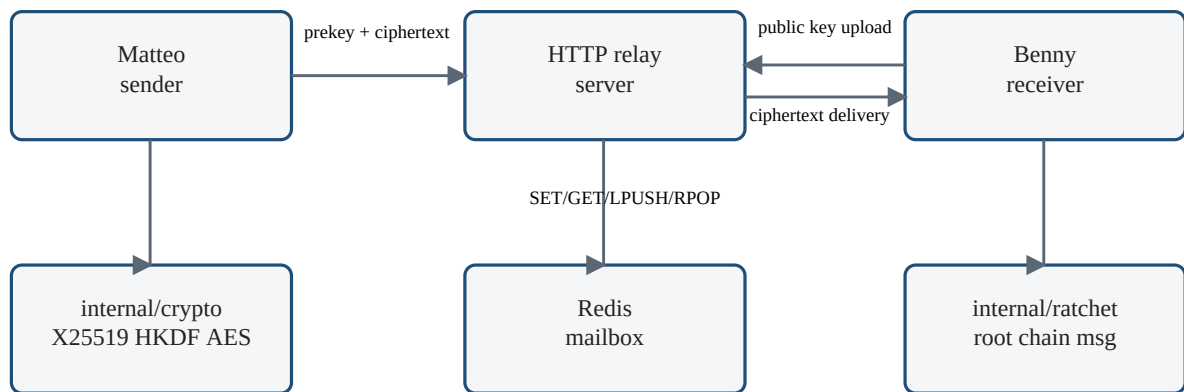
MetaChat demonstrates this idea with a small client-server-client design: Benny is the receiver client, Matteo is the sender client, and the HTTP relay server is backed by Redis. The internal crypto package handles X25519, HKDF, and AES-GCM. The internal ratchet package handles root keys, chain keys, and message keys.

The project name refers to Extended Triple Diffie-Hellman and the Double Ratchet family. The current repository implements a simplified X3DH-inspired handshake. Full X3DH normally includes identity keys, signed prekeys, one-time prekeys, signatures, and identity verification.

Activities

Activity	What was done
Repository design	The project was split into sender, receiver, server, crypto, and ratchet packages.
Key generation	X25519 private and public keys are generated.
Key agreement	Matteo and Benny compute the same shared secret using Diffie-Hellman.
Key derivation	HKDF-SHA256 derives root keys, chain keys, and message keys.
Encryption	AES-256-GCM encrypts and authenticates the plaintext.
Relay server	HTTP endpoints store prekeys and encrypted messages in Redis.
End-to-end run	The server, Benny client, and Matteo client were executed and the decrypted output was checked.

System Architecture



The server is not trusted with plaintext. It stores Benny's public key bundle under `prekey:benny` and encrypted messages under `secure_mailbox:benny`.

Component	File	Responsibility
Crypto engine	<code>internal/crypto/crypto.go</code>	Generates X25519 keys, computes Diffie-Hellman shared secrets, derives HKDF keys, and encrypts/decrypts with AES-GCM.
Ratchet logic	<code>internal/ratchet/ratchet.go</code>	Stores root key and chain key, then advances the chain to produce a message key.
Relay server	<code>server/main.go</code>	Exposes HTTP endpoints and stores data in Redis. It does not decrypt messages.
Benny client	<code>benny/main.go</code>	Uploads the public key, polls the mailbox, derives the key, and decrypts.
Matteo client	<code>matteo/main.go</code>	Reads a message, fetches Benny's key, creates an ephemeral key, encrypts, and sends ciphertext.

Step-by-Step Protocol Walkthrough

1. Benny publishes a public key

Benny generates an X25519 key pair. The private key stays inside Benny's process. The public key is uploaded to the server and stored as `prekey:benny`.

2. Matteo fetches Benny's public key

Matteo calls `GET /prekey?user=benny`. If Benny has not uploaded a key, Matteo stops.

3. Matteo creates an ephemeral key

Matteo creates a fresh key pair for this message. The public part is sent with the encrypted message.

4. Both sides compute the same shared secret

```
S = DH(matteoPriv, bennyPub)
S = DH(bennyPriv, matteoPub)
```

5. The ratchet derives a message key

```
root_key = HKDF(shared_secret, "root")
chain_key = HKDF(root_key, "chain")
chain_key = HKDF(chain_key, "chain-step")
msg_key = HKDF(chain_key, "msg")
```

6. Matteo encrypts and sends

Matteo sends the ephemeral public key, nonce, and ciphertext. The server queues the JSON object in `secure_mailbox:benny`.

7. Benny fetches and decrypts

Benny polls the mailbox, computes the same message key, and decrypts the ciphertext.

Real Code Evidence

X25519 key exchange

```
func GenerateKeyPair() (priv, pub []byte) {
    priv = make([]byte, 32)
    rand.Read(priv)
    pub, _ = curve25519.X25519(priv, curve25519.Basepoint)
    return
}

func DH(priv, pub []byte) []byte {
    shared, _ := curve25519.X25519(priv, pub)
    return shared
}
```

HKDF and AES-GCM

```
func HKDF(secret []byte, info string) []byte {
    h := hkdf.New(sha256.New, secret, nil, []byte(info))
    out := make([]byte, 32)
    io.ReadFull(h, out)
    return out
}

func Encrypt(key, plaintext []byte) (nonce, ciphertext []byte) {
    block, _ := aes.NewCipher(key)
    gcm, _ := cipher.NewGCM(block)
    nonce = make([]byte, gcm.NonceSize())
    rand.Read(nonce)
    ciphertext = gcm.Seal(nil, nonce, plaintext, nil)
    return
}
```

Ratchet message key

```
func (r *Ratchet) NextMessageKey() []byte {
    r.ChainKey = crypto.HKDF(r.ChainKey, "chain-step")
    return crypto.HKDF(r.ChainKey, "msg")
}
```

Matteo sender flow

```
matteoPriv, matteoPub := crypto.GenerateKeyPair()

shared := crypto.DH(matteoPriv, bundle.IdentityKey)
r := ratchet.NewRatchet(shared, bundle.IdentityKey)
```

```

msgKey := r.NextMessageKey()

nonce, ciphertext := crypto.Encrypt(msgKey, []byte(message))

```

Benny receiver flow

```

shared := crypto.DH(bennyPriv, msg.EphemeralKey)
r := ratchet.NewRatchet(shared, msg.EphemeralKey)
msgKey := r.NextMessageKey()

plaintext := crypto.Decrypt(msgKey, msg.Nonce, msg.Ciphertext)

```

Results & Discussion

Package check

The repository compiled successfully with `go test ./...`. The project does not currently include automated unit tests.

```

?      metachat/benny      [no test files]
?      metachat/internal/crypto [no test files]
?      metachat/internal/ratchet [no test files]
?      metachat/matteo    [no test files]
?      metachat/server    [no test files]

```

End-to-end demo output

Redis was not installed on this machine, so the real Go programs were run against a small local Redis-compatible test relay that supports only the commands used by this project. The Go project code was not changed.

```

[SERVER] Running on :8080 (Redis: localhost:6379)
[SERVER] Stored prekey bundle for benny (Redis key: prekey:benny)
[SERVER] Served prekey bundle for benny
[SERVER] Queued secure message for benny (Redis list: secure_mailbox:benny)
[SERVER] Delivered 1 secure message to benny

[MATTEO] Message from CLI args: Hello Benny, this is a secure MetaChat demo for the
        university report.
[MATTEO] Fetching Benny prekey from server...
[MATTEO] Benny prekey received.
[MATTEO] Generating Matteo ephemeral X25519 keypair...
[MATTEO] Computing shared secret (X25519 DH)...
[MATTEO] Initializing ratchet and deriving message key...
[MATTEO] Encrypting message with AES-GCM...
[MATTEO] Sending encrypted message to server (queued for benny in Redis)...
[MATTEO] Done. Message sent.

[BENNY] Generating Benny X25519 identity keypair...
[BENNY] Uploading Benny public key to server (stored in Redis as prekey:benny)...
[BENNY] Benny ready. Polling mailbox from server (secure_mailbox:benny in Redis)...
[BENNY] Received encrypted message. Deriving shared secret (X25519 DH)...
[BENNY] Initializing ratchet and deriving message key...
[BENNY] Decrypting with AES-GCM...
[BENNY] Benny received: Hello Benny, this is a secure MetaChat demo for the university
        report.

```

The result is correct. The server logs show only key storage, ciphertext queueing, and delivery. The plaintext appears only in Matteo's local input and Benny's final output.

What worked well

- The project has clear files and responsibilities.
- The server does not need private keys or plaintext.
- X25519 and AES-GCM are good choices for a small secure messaging demo.

- Redis lists match the idea of a mailbox.
- The protocol is easy to follow for learning.

Limitations

Limitation	Explanation
Simplified X3DH	The code does not yet include signed prekeys, one-time prekeys, or signatures.
Partial Double Ratchet	The ratchet creates message keys, but full send/receive state and skipped keys are not implemented.
No identity verification	Users do not compare fingerprints or safety numbers.
No replay protection	There are no message counters or replay checks.
No TLS	The relay uses plain HTTP.
Crypto errors ignored	Some errors from randomness, X25519, and AES-GCM are not checked.
No key persistence	Keys are regenerated each run.

Conclusion

MetaChat successfully demonstrates the core idea of end-to-end encrypted messaging. Matteo and Benny can derive the same message key without sending the key to the server. The server is useful for delivery, but it does not decrypt messages.

The project is a good academic prototype because the important parts are visible: key generation, Diffie-Hellman, HKDF, ratcheting, AES-GCM, and a mailbox server. The next step would be to complete the X3DH and Double Ratchet design with signed prekeys, one-time prekeys, identity verification, replay protection, and stronger error handling.

Appendix A: Modern Messengers Today

Most modern messengers use a server for routing and a client-side protocol for message content security. Signal documents X3DH for asynchronous key agreement and the Double Ratchet for fresh message keys. WhatsApp and Messenger use Signal Protocol ideas for end-to-end encrypted personal communication, and Meta has published work on key transparency. Telegram is different: normal cloud chats are server-synced, while Secret Chats are the end-to-end encrypted mode and are device-specific.

MetaChat is a small version of these ideas. It publishes public keys, relays encrypted messages, derives message keys on clients, and avoids giving the server plaintext.

Appendix B: API Endpoints

Method	Endpoint	Query	Purpose
POST	/upload_prekey	user	Upload a user public key bundle.
GET	/prekey	user	Fetch a user public key bundle.

Method	Endpoint	Query	Purpose
POST	/send_secure	to	Queue an encrypted message.
GET	/fetch_secure	user	Pop the next encrypted message.

Appendix C: Redis Keys

Redis key	Type	Meaning
prekey:	String	JSON public key bundle.
secure_mailbox:	List	Encrypted messages waiting for the user.

References

- Signal, The X3DH Key Agreement Protocol: <https://signal.org/docs/specifications/x3dh/>
- Signal, The Double Ratchet Algorithm: <https://signal.org/docs/specifications/doubleratchet/>
- Meta Engineering, Deploying key transparency at WhatsApp: <https://engineering.fb.com/2023/04/13/security/whatsapp-key-transparency/>
- Telegram, End-to-End Encryption, Secret Chats: <https://core.telegram.org/api/end-to-end>
- Go package documentation, golang.org/x/crypto/curve25519: <https://pkg.go.dev/golang.org/x/crypto/curve25519>